

National University of Science and Technology POLITEHNICA,
Bucharest

Fetițele Powerpuff - Project Evaluation

Secure Medical OCR Platform with AI-Governed CI/CD

Andrei-Cristian Airinei
Ecaterina-Ioana Mincă
Maria Sfirăială
Maria Teodor
Valentin-Ionuț Bobaru

Team Lead and Software Engineer
Security Software Engineer
Android Software Developer
Security Software Engineer
Security Software Engineer

Secure Systems

1 Project Summary

1.1 Overview

This project is a web platform designed with security in mind, with the purpose of using optical character recognition (OCR) for automated processing of medical documents such as occupational health and safety certificates. Since the system processes protected health information (PHI), the traditional, perimeter-only defense model is replaced by a comprehensive Threat Analysis and Risk Assessment (TARA) framework. The entire software development lifecycle (SDLC) is driven by an AI-Assisted, Security-Optimized CI/CD Architecture, enforcing an absolute human-in-the-loop (HITL) deterministic control gateway before any code modifications enter production. The system captures images from mobile clients, sends them through MQTT over mTLS, processes OCR in an isolated service, stores encrypted medical fields in PostgreSQL, and exposes role-protected reporting APIs and dashboards.

1.2 OSSF Criticality Score

A run of the OSSF Criticality Score tool resulted in a score of 0.22/1, where 0 means least critical and 1 the most critical. This suggests that our Github project has a good score and has a low risk.

2 Functionality

2.1 Data Ingestion and Secure APIs

2.1.1 Secure MQTT Broker for Medical Files

The ingestion channel is based on MQTT with transport security and constrained broker behavior. Mobile clients publish captured medical images to dedicated topics, while server-side consumers process messages asynchronously. Broker configuration applies practical anti-abuse controls (connection limits, queue limits, inflight limits, and payload size restrictions) to reduce denial-of-service impact while preserving near real-time delivery.

Implemented backend topic map:

- `ssproject/images/#` for incoming device images.
- `register/#` for device registration payloads.
- `auth/login/#` for MQTT login requests.
- `device/id/#` for disconnect/inactive updates.
- `ssproject/commands` for command publishing (CAPTURE, START-LIVE, STOP-LIVE).

Example end-to-end usage:

1. Device performs HTTP or MQTT login and obtains a JWT.
2. Device sends registration payload on `register/{device_id}` with token.
3. Backend validates token, links device to `user_email`, and marks the device active.
4. Device publishes image on `ssproject/images/{device_id}`.
5. Backend verifies device ownership before accepting image.

2.1.2 Secure Web Endpoints

The web API exposes authentication, photo management, device information, and analytics/reporting endpoints. Access control is JWT-based, with role-aware behavior for admin-only operations (for example, anonymized export and aggregate reporting views). This enforces a clear separation between regular-user data visibility and privileged analytics capabilities.

2.2 Intelligent OCR Processing

2.2.1 OCR Service Architecture

OCR runs in an isolated service separated from the main API and integrated through service-to-service calls. This separation improves fault containment and allows dedicated hardening of the OCR runtime (restricted container profile, non-root execution, and minimal runtime surface). The pipeline receives document images, extracts structured text fields, and returns normalized outputs for persistence and review. Example OCR processing flow:

1. Incoming image bytes are sent to OCR gRPC `ExtractText`.
2. Backend receives extracted text and marks `ocr_success` based on non-empty output.
3. If OCR fails, text is set to "OCR failed" and success is set to false.

2.2.2 Standardized JSON Output Schema

OCR outputs are normalized into a structured JSON schema used consistently across storage, API responses, and UI rendering. Each extracted field carries both semantic value and review metadata (confidence, manual-edit flag, and validation status), enabling downstream analytics and quality-control workflows without schema conversion. Implemented extracted medical fields include:

- Identity fields: `nume`, `prenume`, `cnp`.
- Employment fields: `societate_unitate`, `profesie_functie`, `loc_de_munca`.
- Control fields: `tip_control` and `control_*` booleans.
- Medical decision fields: `aviz_medical` and `aviz_*` booleans.
- Date fields: `data`, `data_urm_examinari`.

2.2.3 Confidence Thresholding and Manual Review

Low-confidence extraction is handled through a human-in-the-loop flow. Fields below the configured threshold are visually emphasized in the client interface, where operators can edit values and explicitly validate them. This combines automation speed with controlled correction paths for sensitive medical data.

2.3 Dynamic Reporting and Relational Storage

2.3.1 Database Structure

Processed records are stored in PostgreSQL with a hybrid model: relational metadata columns (identifier, timestamp, device/user context, OCR status, latency) plus structured

medical payloads. Sensitive OCR text and medical JSON are encrypted before persistence to support confidentiality requirements while preserving approved reporting capabilities. Storage behavior in implementation:

- OCR text and medical JSON are encrypted before DB save (AES-GCM).
- Data is decrypted on read before response to API clients.
- Image files are stored on disk under `uploads/photos/{id}.jpg`.

2.3.2 Reporting

The reporting layer is implemented as dedicated API endpoints and dashboard widgets.

- Alerts for expired medical check-ups.
- Medical insights and status distribution indicators.
- Performance metrics for OCR processing latency and success rate.
- Anonymized dataset export for controlled sharing.
- Expiry alerts for documents approaching renewal deadlines.
- Fit by profession, overdue checks, and monthly trend analytics.

URGENT EXPIRATION ALERT
16 people have medicina muncii expiring next month.

People affected:

- Ana Florea - 04/06/2026
- Elena Radu - 04/06/2026
- Vlad Ionescu - 05/06/2026
- Ana Popescu - 06/06/2026
- Gabriela Marinescu - 09/06/2026
- Florin Ionescu - 14/06/2026
- Gabriela Gheorghe - 16/06/2026
- Gabriela Radu - 16/06/2026

Remind me later (24h)

Figure 1: Alert for expiring occupational medicine check-ups

Statistics

Start Date

End Date

04/25/2026

05/25/2026

Refresh

Download Anonymized Report

Figure 2: Anonymized dataset generation button

```
{
  "generated_at": "2026-05-25T09:15:28.794187358Z",
  "count": 80,
  "documents": [
    {
      "document_id": "8b973e7e-2dc3-472e-83a4-c2a0490ed25f",
      "timestamp": "2026-05-31T11:08:00Z",
      "image_type": "jpeg",
      "medical_data": {
        "tip_control": {
          "value": "Control Supraveghere",
          "confidence": 0.95,
          "is_edited": false,
          "is_validated": false
        },
        "control_angajare": {
          "value": false,
          "confidence": 0.95,
          "is_edited": false,
          "is_validated": true
        },
        "control_periodic": {
          "value": false,
          "confidence": 0.98,
          "is_edited": false,
          "is_validated": true
        },
        "control_adaptare": {
          "value": false,
          "confidence": 1,
          "is_edited": false,
          "is_validated": false
        },
        "control_reluare": {
          "value": false,
          "confidence": 0.96,
```

Figure 3: Anonymized dataset example

2.4 Mobile Functionalities

2.4.1 Image Capture Using Device Camera

The Android client uses the device camera to capture medical documents directly from the field. Capture settings are exposed in the UI to support quality/throughput trade-offs, and images are prepared for immediate secure upload or deferred transmission in unstable network conditions. Example usage:

1. User logs in from Setup screen (email/password) and configures broker IP/port.
2. User enables Live Mode or presses Capture.
3. Camera frame is converted to JPEG and sent to MQTT topic for that device.

2.4.2 Secure MQTT Transmission with mTLS

Mobile-to-platform transport uses MQTT with optional mTLS configuration, combined with authenticated session handling. Connection setup includes broker endpoint configuration, credential-based login flow, and controlled reconnect behavior. This supports secure edge ingestion in both local testing and deployment-like environments. Authentication and registration flow:

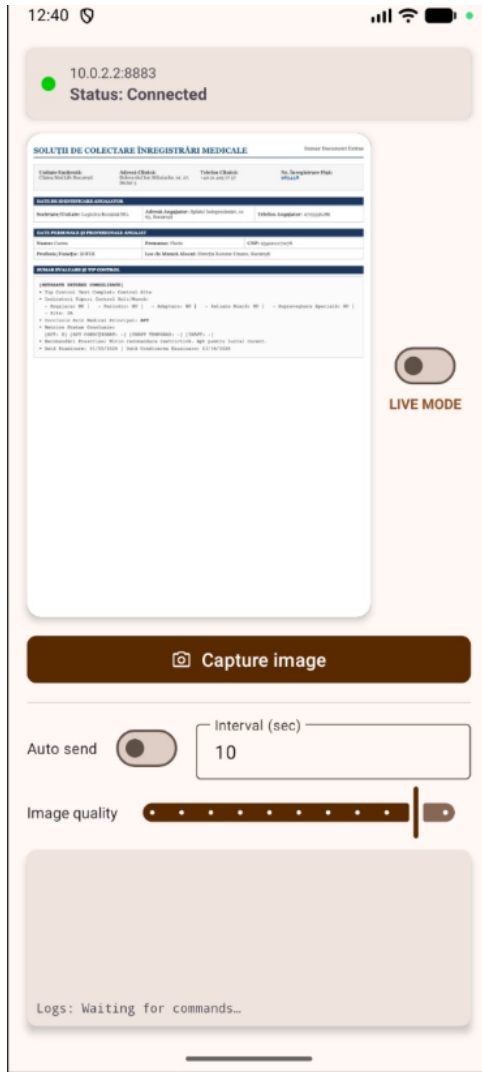
1. Mobile app performs HTTP login to server `/login` and obtains JWT.
2. After MQTT connect, app publishes registration payload to `register/{device_id}` with token.
3. Server validates token and associates device with user before accepting images.

2.4.3 Offline Mode

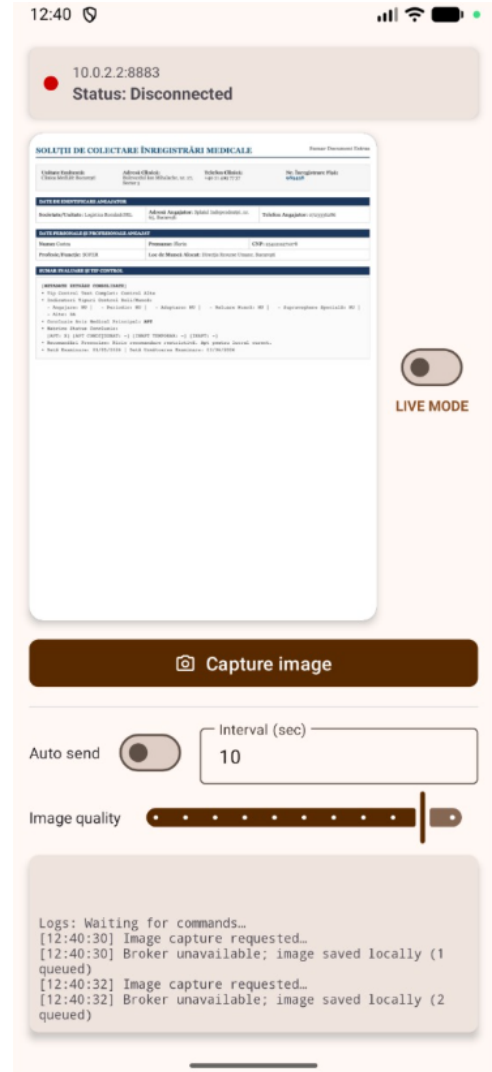
When connectivity is unavailable, the application persists pending images in a bounded local queue. A background retry/flush mechanism attempts transmission after reconnection, preserving data continuity and minimizing operator intervention. Runtime logs and status indicators expose whether each image was sent live or queued offline.

Example usage:

1. Broker/network becomes unavailable during capture.
2. New captures are queued locally instead of being dropped.
3. When connection returns, app automatically retries and removes images from queue after successful send.



(a) Connected mode



(b) Offline mode

Figure 4: Android mobile capture states

2.5 Dark Mode

Dark mode support improves usability in low-light conditions and long-running monitoring sessions. The interface theme can be switched without affecting workflow semantics, and key status colors (alerts, warnings, success indicators, and confidence cues) remain distinguishable in both light and dark modes.



Figure 5: Example of light mode page

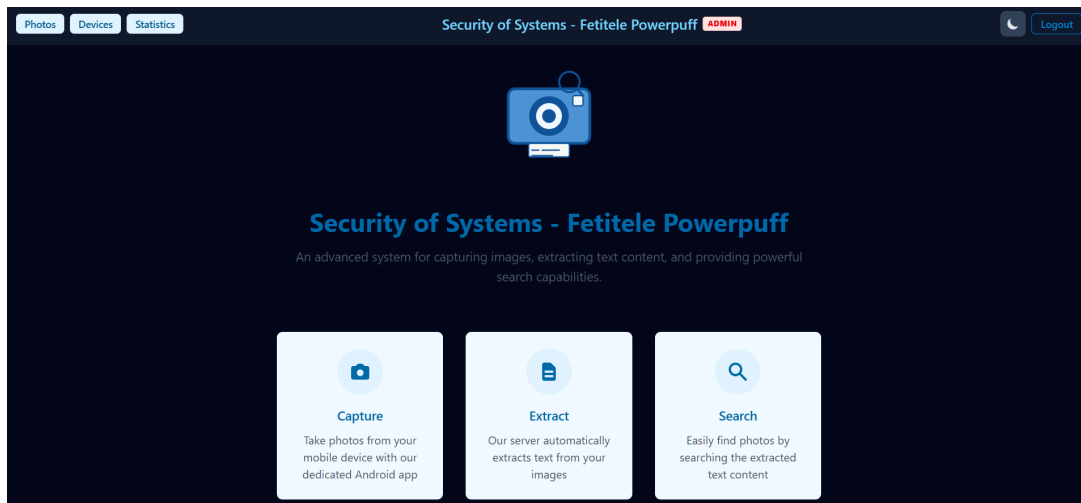


Figure 6: Example of dark mode page

3 Documentation

3.1 Developer Documentation

3.1.1 System Overview

The web platform is composed of five main components:

- React + Vite frontend in `web/client`.
- Go backend API in `web/server`.
- Go gRPC OCR microservice in `web/ocr-service`.
- Mosquitto MQTT broker in `web/broker`.

- PostgreSQL persistence layer.

The operational data flow starts from mobile capture or web upload, continues through the backend and OCR processing pipeline, then passes through structured medical parsing and encrypted storage, and finally reaches API responses and frontend visualization.

3.1.2 Security Architecture

Transport security is enforced end-to-end: MQTT communication runs over TLS on port 8883 with client certificate validation, backend-to-OCR calls use gRPC with mTLS, and web API access is protected through JWT bearer tokens. The broker is hardened against abuse using concrete limits from Mosquitto configuration:

- Payload size cap: 10 MB.
- Maximum concurrent connections: 100.
- Maximum inflight messages per client: 20.
- Maximum queued messages per client: 100.

JWT middleware reads the `Authorization: Bearer` header, verifies signature and expiration, then injects identity attributes (email and role) into request context for downstream authorization.

Role boundaries are explicit:

- Admin-only examples:
 - `GET /users`
 - `DELETE /photos/all`
 - `GET /photos/anonymized/export`
- Non-admin users are scoped to their own records.

3.1.3 Backend Structure

The backend is organized into focused modules:

- `routes`: HTTP route handlers.
- `broker`: MQTT message handlers.
- `repository`: data access through GORM.
- `domain`: core models (User, Device, Photo, MedicalData).
- `utils`: authentication, storage, medical parsing helpers.
- `ocr`: gRPC client wrapper.

Main API groups:

- Authentication:
 - POST /register
 - POST /login
 - GET /profile
 - GET /users (admin)
- Devices:
 - GET /devices
 - POST /devices/claim
 - POST /devices/switch
 - POST /devices/command
- Photos and analytics:
 - GET /photos
 - POST /photos
 - PATCH /photos/{id}
 - DELETE /photos/{id}
 - DELETE /photos/all (admin)
 - GET /photos/performance
 - GET /photos/medical-insights
 - GET /photos/anonymized/export (admin)
- System helper:
 - GET /broker-info

3.1.4 Database and Storage

The data model is auto-migrated around three principal entities:

- User
- Device
- Photo

Photo persistence follows a split-storage strategy where image binaries are stored on disk under `uploads/photos/{photo_id}.jpg`, while metadata and structured state are kept in PostgreSQL.

Sensitive data is protected at rest using AES-GCM encryption:

- OCR free-text content is persisted as `text_enc`.
- Structured medical content is persisted as `medical_data_enc`.
- Decryption is performed in the repository layer at read time before API response serialization.

3.1.5 OCR and Structured Extraction

The OCR service is implemented as a Go gRPC server listening on port 50051 and using Gosseract/Tesseract with English and Romanian language packs. OCR responses include:

- `text`
- `processing_time_ms`
- `error` (when relevant)

After text extraction, the backend checks whether the content resembles a medicina muncii certificate and then maps recognized values into a structured `MedicalData` payload. Each extracted field carries:

- a typed value,
- confidence (0.0 to 1.0),
- optional review flags `is_edited` and `is_validated`.

Confidence-based review logic in the web UI marks fields for manual review when confidence is below 0.95 and the field is neither edited nor validated. Users can:

- validate individual fields,
- validate all fields,
- edit values (edited fields are treated as validated).

Review changes are persisted through `PATCH /photos/{id}`.

3.1.6 Statistics and Reporting

The statistics module combines record-level and aggregate reporting for both operational monitoring and medical compliance review. Core analytics include OCR performance metrics, medical insight summaries, and role-aware report endpoints surfaced in the dashboard. Date-range filtering is supported in reporting calls, enabling bounded views over selected periods.

Implemented report capabilities include:

- fit-by-profession analysis,

- overdue medical check detection,
- monthly compliance trend with completed, fit, overdue, and expiring-next-month series,
- anonymized dataset export,
- fitness diagnosis by profession

The frontend presents these datasets in chart and table form, while anonymized dataset export supports privacy-preserving sharing for evaluation and external analysis. Admin-only access is preserved for destructive or sensitive operations, and non-admin users see data scoped to their own records.

3.2 User Documentation

3.2.1 What the Application Does

The application helps users capture and upload medical documents, extract key information automatically, review unclear values, monitor activity through statistics, and manage connected capture devices. Users with administrator rights can also access organization-wide views and export anonymized reports.

3.2.2 Getting Access

Create an account from the Register page, then sign in from the Login page using your email and password. After login, protected pages become available. Regular users see their own data, while administrators can access global data and additional management actions.

3.2.3 Mobile App Guide

At first launch, users configure connection settings and sign in. The mobile dashboard supports manual capture, continuous live capture, periodic automatic send, and image-quality adjustments. If connectivity drops, the app keeps new captures safely in a local queue and sends them automatically when the connection is restored.

3.2.4 Devices Page

The Devices page is used to monitor connected devices and trigger capture actions remotely. Users can start a capture, start live mode, or stop live mode, and can also verify current connection state.

3.2.5 Photos Page

The Photos page allows searching and filtering records, uploading new images, and opening entries for review. Users can inspect extracted fields, validate values, and correct

information when needed. Individual photos can be removed, while full bulk deletion remains an admin-only operation.

3.2.6 Statistics Page

The Statistics page presents activity summaries, medical-status distributions, performance indicators, and renewal-related insights. Date filters can be used to focus results on a selected period. If upcoming expirations are detected, an alert is displayed and can be snoozed temporarily.

3.2.7 Anonymized Report Export

Administrators can download anonymized reports for analysis and presentation. These exports are intended for sharing trends and outcomes without exposing direct personal identifiers.

4 Execution

4.1 CI/CD

4.1.1 Web CI Workflow

The web pipeline is implemented in `.github/workflows/ci-web.yml` (workflow name: CI). It is triggered on push to `main`, pull requests targeting `main`, and through `workflow_call` so it can be reused by other workflows.

Main validation steps include:

- Go backend build and tests in `web/server`.
- Frontend lint/build in `web/client`.
- Docker build for the API image.
- Artifact upload for `go-test-results` and `go-coverage`.

4.1.2 Mobile CI/CD Workflow

The mobile pipeline is implemented in `.github/workflows/ci-mobile.yml` (workflow name: Mobile CI/CD). It is triggered on push to `main`, push on version tags (`v*`), pull requests targeting `main`, and `workflow_call`. Path filters include the mobile module and mTLS generation scripts.

The `mobile_checks` job performs JDK 17 setup, Android mTLS resource generation, lint, Semgrep static analysis, unit tests, instrumented tests on an emulator (API 31), and debug APK build, then uploads `mobile-test-reports` and `app-debug-apk` artifacts. The `release` job runs only for tag refs, downloads the APK artifact, and publishes it in a GitHub Release.

4.1.3 CodeQL Advanced Workflow

CodeQL analysis is implemented in `.github/workflows/codeql.yml` (workflow name: CodeQL Advanced), triggered on push to `main` and pull requests targeting `main`. The language matrix includes Go on autobuild and Java/Kotlin on manual build path.

The workflow initializes CodeQL with security-extended and security-and-quality query suites, prepares Android prerequisites where needed, runs analysis, and publishes code scanning results.

4.1.4 SBOM Update Automation

SBOM maintenance is automated in `.github/workflows/update-sbom-readme.yml` (workflow name: Update SBOM README). It runs on `main` pushes that match dependency/SBOM path filters and can also be launched manually via `workflow_dispatch`.

The workflow executes `web/scripts/update-sbom-readme.sh`, regenerates `web/server/sbom.cdx.json` and `web/ocr-service/sbom.cdx.json`, updates the SBOM snapshot in the root README, and commits/pushes changes automatically when differences are detected.

4.1.5 AI Governance and Quarantine Promotion

AI governance and quarantine promotion are intentionally coupled in the delivery process. Governance rules (architectural guardrails, RBAC-like zone restrictions, forbidden-path boundaries, and audit logging) constrain what AI-assisted changes can modify and require traceability for every automation step.

Promotion is implemented in `.github/workflows/promote-quarantine.yml` (workflow name: AI Quarantine Promotion), triggered on pushes to `quarantine/**`. The workflow calls reusable web and mobile validation workflows (`ci-web.yml` and `ci-mobile.yml`). Only when these validations pass does the automation create/promote the PR to `main`. Audit evidence from `ai-governance/audit.log` is attached to the PR body, preserving a human-reviewable governance trail.

The flow of operations is aligned according to the following steps:

1. Plan: AI orchestrator receives task and logs rationale.
2. Govern: Capability gateway enforces write zones and forbidden paths.
3. Execute: Changes are staged on `quarantine/*` branches and validated by CI workflows.
4. Verify: Web and mobile pipelines must pass before automated PR promotion.
5. Record: Audit log is attached in promotion PR body for traceability evidence.

This ensures the practice of human-in-the-loop, risk to main branch is reduced by quarantine measures, while artifacts and workflow logs provide traceability required for auditing.

5 Security and Compliance

5.1 Threat Modeling and Mitigations

Threats such as untrusted image payloads, OCR RCE risk, PHI leakage in transit and at rest, unauthorized access to medical records, CI/CD bypass by autonomous agents and DoS risk on MQTT ingestion were addressed through numerous security measures. Mitigations include the following, as well as many others:

- OCR service runs in a constrained container setup with read-only root, tmpfs, no-new-privileges, non-root runtime, and distroless final image.
- mTLS is used for MQTT and OCR gRPC service-to-service communication.
- HTTPS endpoint is exposed for login/API access.
- Medical text and structured medical data are encrypted before DB persistence (AES-GCM service in repository layer).
- JWT-based authentication and role checks are used on protected routes.
- MQTT broker limits include:
 - `message_size_limit`
 - `max_connections`
 - `max_inflight_messages`
 - `max_queued_messages`
- AI governance enforces RBAC-style write zones and quarantine branch strategy.

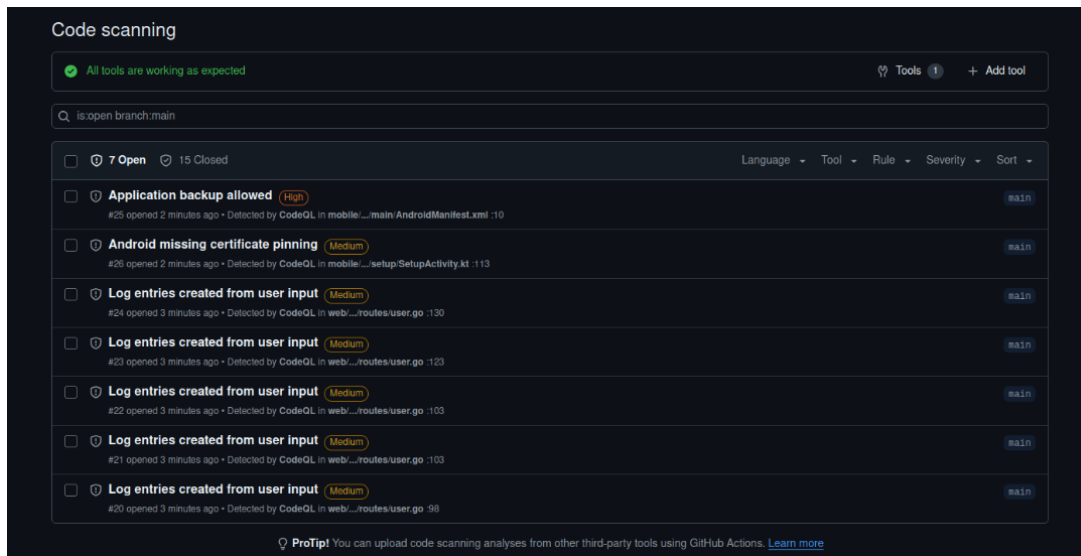


Figure 7: CodeQL Output

In order to automate threat modeling, CodeQL tool was used for static code analysis. At every new PR, the tool runs for each line added. CodeQL generates and reports

issues. In figure 7, it can be seen that the Pull Requests have certain issues. The Static Analysis Summary is provided by the CodeQL tool and it uses the security-extended and security-and-quality query packs. We use this tool because it is more suitable for the go language. Not only that, but it found an Easter egg that could turn critical for Kotlin too.

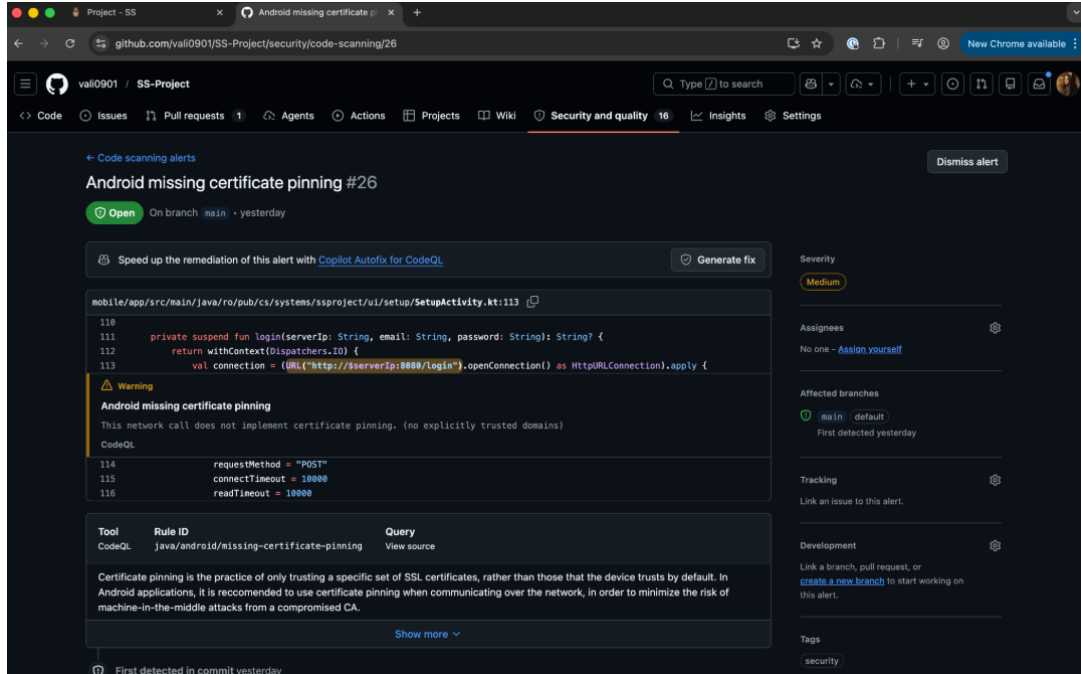


Figure 8: Kotlin issue reported by CodeQL

5.2 MISRA / CERT Compliance

Project languages are primarily Go, TypeScript, Kotlin, and Python. MISRA C is not directly applicable to the main codebase. Instead, we adapted by using more suitable solutions, such as CodeQL for advanced security analysis for Go and Java/Kotlin, Semgrep static checks for mobile pipeline. Standards such as non-root containers, key/cert secret mounts, and explicit TLS validation, password hashing with bcrypt and token validation middleware, were also upheld.

5.3 Testing and Coverage

5.3.1 Testing

Testing is accomplished on all fronts, the project encompassing Go backend unit tests for routes, auth, broker handlers, performance/report logic, as well as Android unit tests and instrumented tests in CI and frontend lint/build checks in CI.

5.3.2 Coverage

mqtt-streaming-server/repository/user_repository.go:30:	FindByEmail	0.0%
mqtt-streaming-server/repository/user_repository.go:42:	GetAll	0.0%
mqtt-streaming-server/routes/device.go:20:	InitDeviceRoutes	100.0%
mqtt-streaming-server/routes/device.go:32:	ClaimDevice	100.0%
mqtt-streaming-server/routes/device.go:73:	SwitchDeviceMode	100.0%
mqtt-streaming-server/routes/device.go:113:	GetDevices	100.0%
mqtt-streaming-server/routes/device.go:142:	SendCommand	100.0%
mqtt-streaming-server/routes/init.go:17:	InitRoutes	0.0%
mqtt-streaming-server/routes/init.go:38:	handleBrokerInfo	100.0%
mqtt-streaming-server/routes/init.go:57:	getOutboundIP	71.4%
mqtt-streaming-server/routes/init.go:87:	withCORS	100.0%
mqtt-streaming-server/routes/init.go:105:	withAuth	87.5%
mqtt-streaming-server/routes/photo.go:32:	InitPhotoRoutes	100.0%
mqtt-streaming-server/routes/photo.go:51:	HandlePhotoByID	100.0%
mqtt-streaming-server/routes/photo.go:62:	GetPhotos	100.0%
mqtt-streaming-server/routes/photo.go:134:	UploadPhoto	80.4%
mqtt-streaming-server/routes/photo.go:226:	UpdatePhoto	100.0%
mqtt-streaming-server/routes/photo.go:278:	DeletePhoto	100.0%
mqtt-streaming-server/routes/photo.go:323:	DeleteAllPhotos	100.0%
mqtt-streaming-server/routes/user.go:20:	InitUserRoutes	100.0%
mqtt-streaming-server/routes/user.go:31:	GetUsers	100.0%
mqtt-streaming-server/routes/user.go:60:	Register	100.0%
mqtt-streaming-server/routes/user.go:140:	Login	88.9%
mqtt-streaming-server/routes/user.go:190:	GetProfile	100.0%
mqtt-streaming-server/utils/auth.go:19:	VerifyToken	66.7%
mqtt-streaming-server/utils/auth.go:38:	GenerateToken	100.0%
mqtt-streaming-server/utils/auth.go:51:	CheckPassword	100.0%
mqtt-streaming-server/utils/auth.go:55:	HashPassword	100.0%
mqtt-streaming-server/utils/medical_parser.go:12:	IsMedicalCertificate	100.0%
mqtt-streaming-server/utils/medical_parser.go:28:	ParseMedicalCertificate	100.0%

Figure 9: Test coverage (1)

mqtt-streaming-server/repository/user_repository.go:30:	FindByEmail	0.0%
mqtt-streaming-server/repository/user_repository.go:42:	GetAll	0.0%
mqtt-streaming-server/routes/device.go:20:	InitDeviceRoutes	100.0%
mqtt-streaming-server/routes/device.go:32:	ClaimDevice	100.0%
mqtt-streaming-server/routes/device.go:73:	SwitchDeviceMode	100.0%
mqtt-streaming-server/routes/device.go:113:	GetDevices	100.0%
mqtt-streaming-server/routes/device.go:142:	SendCommand	100.0%
mqtt-streaming-server/routes/init.go:17:	InitRoutes	0.0%
mqtt-streaming-server/routes/init.go:38:	handleBrokerInfo	100.0%
mqtt-streaming-server/routes/init.go:57:	getOutboundIP	71.4%
mqtt-streaming-server/routes/init.go:87:	withCORS	100.0%
mqtt-streaming-server/routes/init.go:105:	withAuth	87.5%
mqtt-streaming-server/routes/photo.go:32:	InitPhotoRoutes	100.0%
mqtt-streaming-server/routes/photo.go:51:	HandlePhotoByID	100.0%
mqtt-streaming-server/routes/photo.go:62:	GetPhotos	100.0%
mqtt-streaming-server/routes/photo.go:134:	UploadPhoto	80.4%
mqtt-streaming-server/routes/photo.go:226:	UpdatePhoto	100.0%
mqtt-streaming-server/routes/photo.go:278:	DeletePhoto	100.0%
mqtt-streaming-server/routes/photo.go:323:	DeleteAllPhotos	100.0%
mqtt-streaming-server/routes/user.go:20:	InitUserRoutes	100.0%
mqtt-streaming-server/routes/user.go:31:	GetUsers	100.0%
mqtt-streaming-server/routes/user.go:60:	Register	100.0%
mqtt-streaming-server/routes/user.go:140:	Login	88.9%
mqtt-streaming-server/routes/user.go:190:	GetProfile	100.0%
mqtt-streaming-server/utils/auth.go:19:	VerifyToken	66.7%
mqtt-streaming-server/utils/auth.go:38:	GenerateToken	100.0%
mqtt-streaming-server/utils/auth.go:51:	CheckPassword	100.0%
mqtt-streaming-server/utils/auth.go:55:	HashPassword	100.0%
mqtt-streaming-server/utils/medical_parser.go:12:	IsMedicalCertificate	100.0%
mqtt-streaming-server/utils/medical_parser.go:28:	ParseMedicalCertificate	100.0%

Figure 10: Test coverage (2)

The go cover tool is used for finding the test coverage of the project. We tried to focus on the most important sections to reach a high score in order to obtain a more secure project. As it can be seen in the pictures above, almost every important functionality (device, user and photo) has 100% code coverage testing. We integrated the code coverage test in the CI of the web app. The fuzzing testing is integrated in the codes written for testing that introduce malformed input in order to see that our web app is returning the good error http code (400 bad request).

The coverage results shown in Figures 9 and 10 indicate that the most critical modules have strong automated test coverage.

5.4 SBOM and Dependencies

CycloneDX SBOM is generated for web and OCR services with an automated update workflow which commits refreshed artifacts. The maintained SBOM summary can be used to track dependency inventory for auditing purposes.

5.5 Fixing Own Vulnerabilities

Among the fixed security issues, the following can be counted:

1. Dependabot upgraded vulnerable frontend packages in *yarn.lock*, *react-router*, *react-router-dom*, *minimatch*, *picomatch*, and *set-cookie-parser*.
2. CodeQL-related remediations:
 - Disabled Android app backup and excluded app data from backup/transfer.
 - Replaced Android HTTP login with HTTPS.
 - Added Android network security config for trusted CA and generated certificate pinning.
 - Removed raw user-controlled email values from backend auth logs.

Besides those, the following can be counted towards hardening the application:

1. HTTPS listener for the Go API and enabled Renovate.
2. Encryption-at-rest for OCR text and medical structured fields.
3. OCR isolation with distroless runtime and restrictive container controls.
4. JWT auth + role-gated endpoints for admin-only reports and exports.
5. MQTT broker resource limits to reduce DoS impact.
6. Quarantine-gated AI changes with mandatory CI validation before PR promotion.

5.6 Reporting Peer Issues

Evidence for a reported peer-project issue is included below.

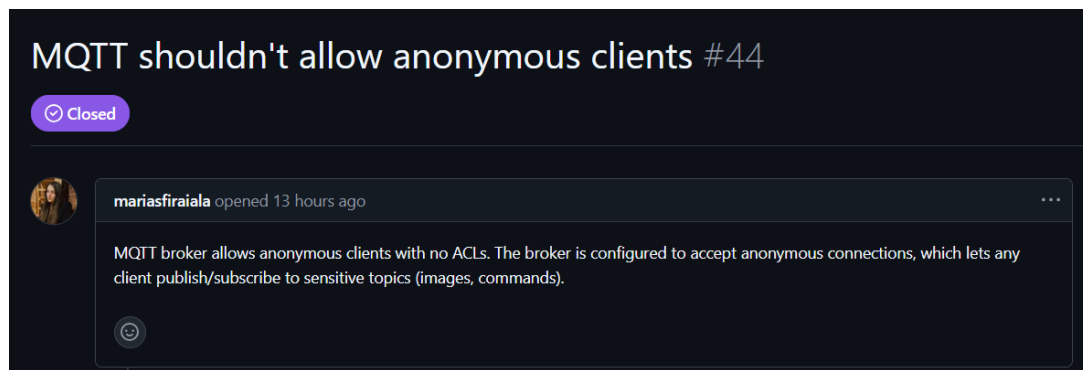


Figure 11: Peer issue report evidence

6 Medical Reports

Report 1: Total Processed Medical Documents

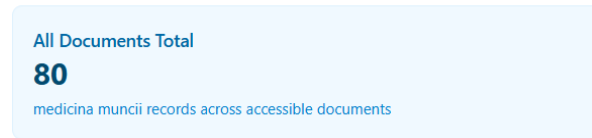


Figure 12: Total of documents submitted

Report 2: Medical Opinion Distribution

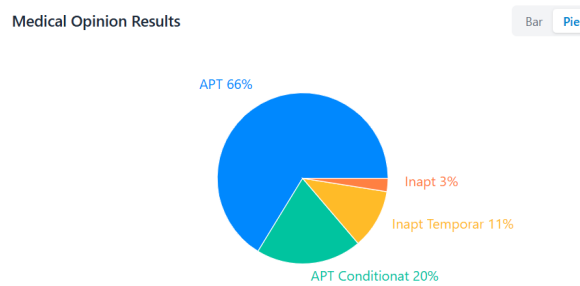


Figure 13: Distribution of medical opinions across documents

Report 3: Control Type Distribution

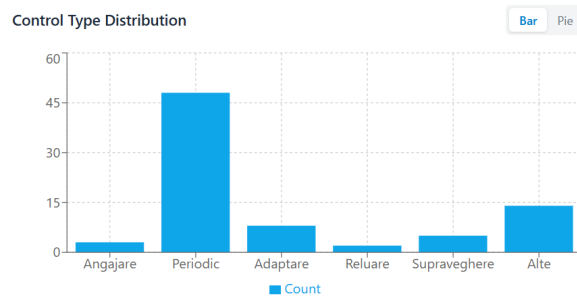


Figure 14: Distribution of control types across data

Report 4: Documents Issued in Last 30 Days

Medicina Muncii Insights

Last Month

20

documents issued in the last 30 days

Figure 15: Total of documents submitted within the last 30 days

Report 5: Expiring Next Month

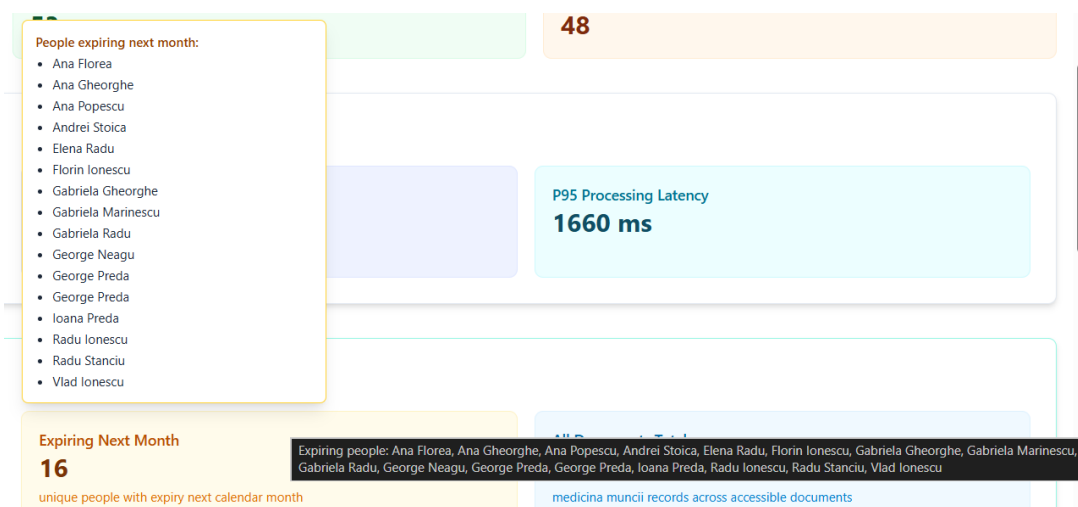


Figure 16: People whose control expiration date is within the next month

Report 6: Overdue Medical Checks

Overdue Medical Checks			
Total Overdue People 13			
Mihai Radu	STUDENT	05/03/2026	81
Daniela Matei	PROFESOR UNIVERSITAR	06/03/2026	80
Ion Costea	INGINER	18/03/2026	68
Andrei Ionescu	CONTABIL	01/04/2026	54
Daniela Dumitru	PROFESOR UNIVERSITAR	01/04/2026	54
Andrei Popescu	PROFESOR UNIVERSITAR	01/05/2026	24
Vlad Preda	PROFESOR UNIVERSITAR	01/05/2026	24
Andrei Stoica	PROFESOR UNIVERSITAR	14/05/2026	11

Figure 17: People with overdue medical checks

Report 7: Fit Rate by Profession



Figure 18: Fit rate distributed by profession

Report 8: Monthly Compliance Trend

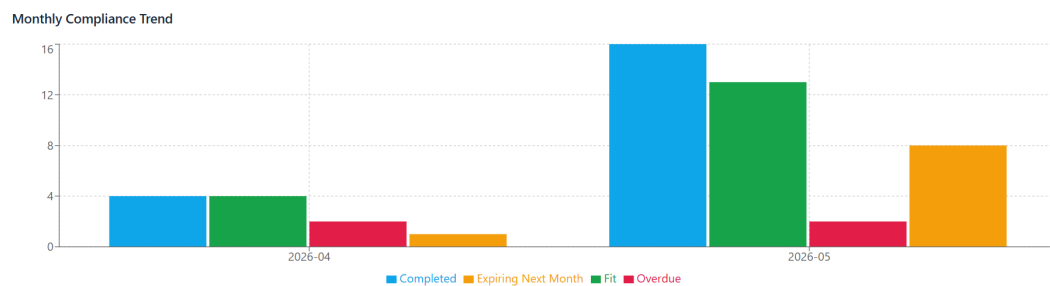


Figure 19: Trend of fit diagnosis distributed monthly

7 Team Contributions

Contribution table based on repository history is the following, bearing in mind that the contributions were not not limited to Github code and may vary based on personal habits such as squashing or not squashing one's commits:

Team Member	Lines Added	Lines Removed	Number of Commits
Andrei Airinei	5961	6896	68
Ecaterina-Ioana Minca	15142	593	29
Maria Teodor	6457	1296	21
Maria Sfraiala	1415	345	15
Valentin-Ionut Bobaru	3257	130	13

Table 1: Git statistics per team member

In order to organize better, tasks were proposed and assigned on a voluntary basis using the Trello platform. Some of the tasks and their assignee can be observed in Figure 20.

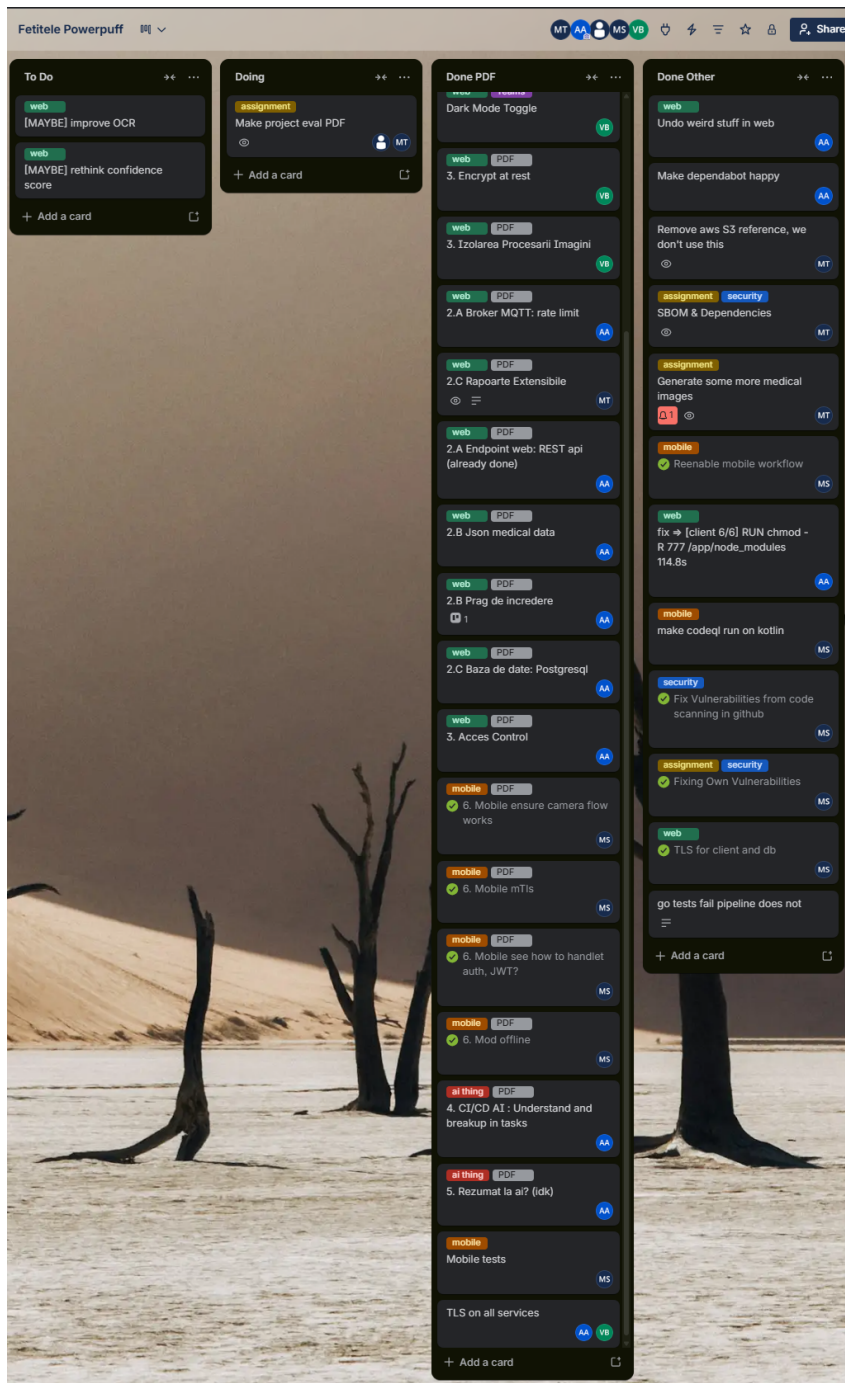


Figure 20: Trello Board for Tasks

8 OSSF Criticality Score

Figure 21 presents a screenshot of the output of running OSSF Criticality score tool using `original_pike.yml`.

```
repo.url: https://github.com/vali0901/SS-Project
repo.language: Go
repo.license:
repo.star_count: 0
repo.created_at: 2026-03-23T18:07:53Z
repo.updated_at: 2026-05-24T21:17:00Z
legacy.created_since: 2
legacy.updated_since: 0
legacy.contributor_count: 7
legacy.org_count: 1
legacy.commit_frequency: 2.63
legacy.recent_release_count: 2
legacy.updated_issues_count: 0
legacy.closed_issues_count: 0
legacy.issue_comment_frequency: 0
legacy.github_mention_count: 0
default_score: 0.22473
```

Figure 21: OSSF Criticality Score